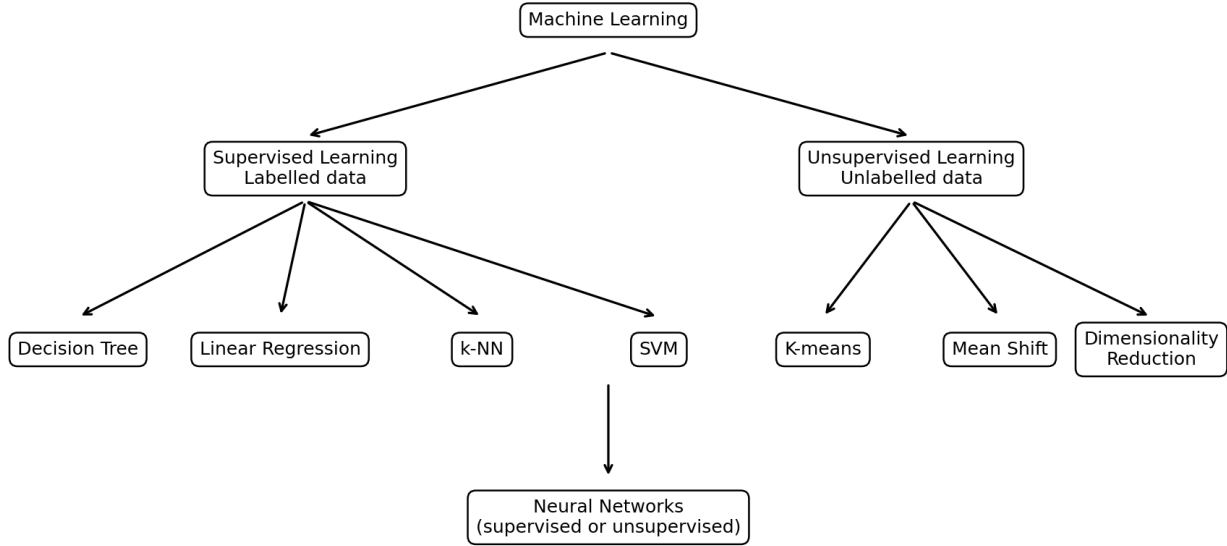


Unit 04 Algorithm Map



DSML LAB PRACTICAL MANUAL

Lab Tasks, Solutions and Step-by-Step Tutorials

Decision Tree | Linear Regression | k-NN | SVM | K-Means | Distance Metrics | Dimensionality Reduction

Designed for classroom demonstration, laboratory practice, practical examination and record-file preparation

Student Name	
Roll Number	
Course / Semester	
Institution	
Academic Session	

Prepared for Data Science and Machine Learning Laboratory

How to Use This Lab Manual

This manual contains seven complete practical experiments. Each experiment follows the same record-file sequence: Aim, Requirements, Theory, Algorithm, Program, Output, Interpretation, Result, Viva Questions and Practice Exercises. Programs have been executed using Python and verified with scikit-learn 1.8.0 and SciPy 1.17.0 in the authoring environment.

Important: The exact accuracy may change when `random_state`, `test_size`, dataset or model parameters are changed. A different but reasonable value is not automatically an error.

Laboratory Learning Outcomes

- Load built-in datasets and separate features from target values.
- Divide data into training and testing sets and prevent data leakage.
- Train supervised classification and regression algorithms.
- Apply unsupervised clustering and interpret cluster quality.
- Calculate commonly used distance measures manually and with SciPy.
- Reduce high-dimensional data with PCA, LDA and t-SNE.
- Evaluate models using confusion matrices, accuracy, MAE, MSE, RMSE, R-squared, inertia and silhouette score.
- Read plots and explain whether an output supports the stated objective.

Software and Installation

Component	Purpose
Python 3.10 or later	Programming language
NumPy	Numerical arrays and calculations
pandas	Tabular data handling
Matplotlib	Plots and visual output
SciPy	Scientific functions and distance metrics
scikit-learn	Datasets, preprocessing, ML algorithms and evaluation

Installation commands

```
# Run in Command Prompt / Terminal
python -m pip install numpy pandas matplotlib scipy scikit-learn

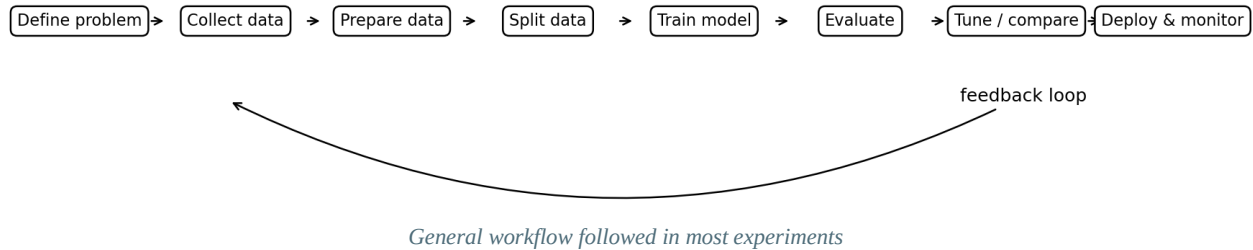
# Verify installation
python -c "import sklearn, scipy; print(sklearn.__version__, scipy.__version__)"
```

Recommended Folder Structure

```
DSML_Lab/
|-- exp01_decision_tree.py
|-- exp02_linear_regression.py
|-- exp03_knn.py
|-- exp04_svm.py
|-- exp05_kmeans.py
|-- exp06_distance_metrics.py
|-- exp07_dimensionality_reduction.py
`-- outputs/
```

General Machine Learning Workflow

A Practical Machine Learning Workflow



1. Define the problem: classification, regression, clustering or visualization.
2. Load and understand the dataset.
3. Preprocess the features, especially scaling for distance-based methods.
4. Split labelled data into training and testing subsets.
5. Create the model with suitable parameters.
6. Train the model with fit().
7. Generate predictions or cluster labels.
8. Evaluate results and interpret limitations.

List of Experiments

No.	Practical	Primary task
1	Decision Tree Algorithm	Classification
2	Linear Regression	Regression
3	k-Nearest Neighbours (k-NN)	Classification
4	Support Vector Machine (SVM)	Classification
5	K-Means Clustering	Unsupervised clustering
6	Various Distance Metrics	Similarity / dissimilarity
7	Dimensionality Reduction Techniques	Feature compression and visualization

EXPERIMENT 1

Decision Tree Classification

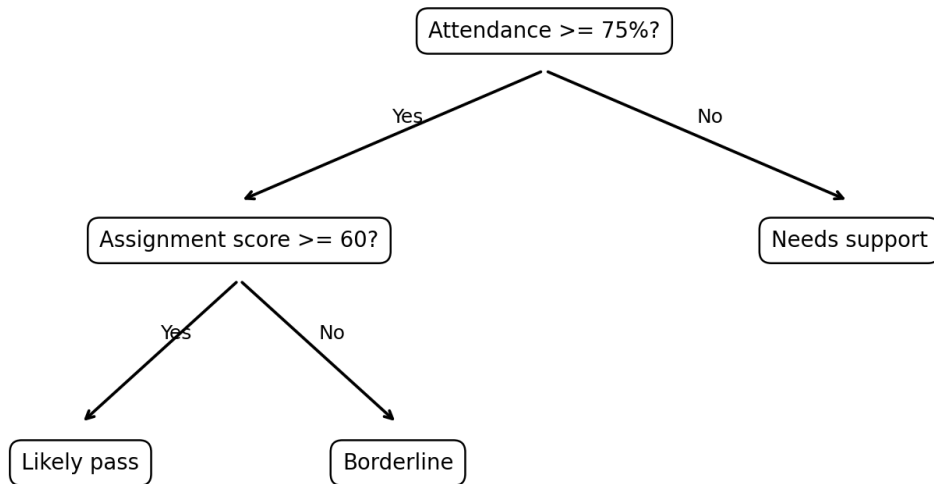
Aim: To implement a Decision Tree classifier using the Iris dataset, predict flower species, visualize the tree and evaluate performance.

Requirements: Python, NumPy, Matplotlib and scikit-learn.

Concept

A Decision Tree makes predictions through a sequence of yes/no questions. An internal node tests a feature, a branch represents an answer and a leaf stores the final class. During training, the algorithm searches for splits that make child nodes purer than the parent node.

A Small Decision Tree for Student Support



Decision Tree concept: tests divide the data into increasingly pure groups

Term	Meaning
Root node	First question applied to all training records
Internal node	A later feature test
Branch	Outcome of a test
Leaf node	Final predicted class
Gini impurity	Measures class mixing; 0 means a completely pure node
max_depth	Limits tree growth and helps control overfitting

Gini impurity: $Gini = 1 - \sum(p_i^2)$, where p_i is the proportion of class i in a node. The algorithm favours splits that reduce impurity.

Algorithm

1. Load the Iris dataset and separate X (features) and y (species labels).
2. Split records into training and testing data using stratification.
3. Create DecisionTreeClassifier with criterion="gini" and max_depth=3.
4. Train the tree on X_train and y_train.
5. Predict the classes of X_test.
6. Calculate accuracy, confusion matrix and classification report.
7. Plot the trained decision tree.

Complete Python program

```

import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree

```

```

from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
)

# 1. Load data
iris = load_iris()
X = iris.data
y = iris.target

# 2. Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.25,
    random_state=42,
    stratify=y,
)

# 3. Create and train model
model = DecisionTreeClassifier(
    criterion="gini",
    max_depth=3,
    random_state=42,
)
model.fit(X_train, y_train)

# 4. Test the model
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:
", confusion_matrix(y_test, y_pred))
print(classification_report(
    y_test, y_pred,
    target_names=iris.target_names,
))

# 5. Predict one new flower
new_flower = [[5.1, 3.5, 1.4, 0.2]]
prediction = model.predict(new_flower)[0]
print("New flower:", iris.target_names[prediction])

# 6. Visualize the tree
plt.figure(figsize=(12, 7))
plot_tree(
    model,
    feature_names=iris.feature_names,
    class_names=iris.target_names,
    filled=True,
    rounded=True,
)
plt.title("Decision Tree - Iris Dataset")
plt.tight_layout()
plt.show()

```

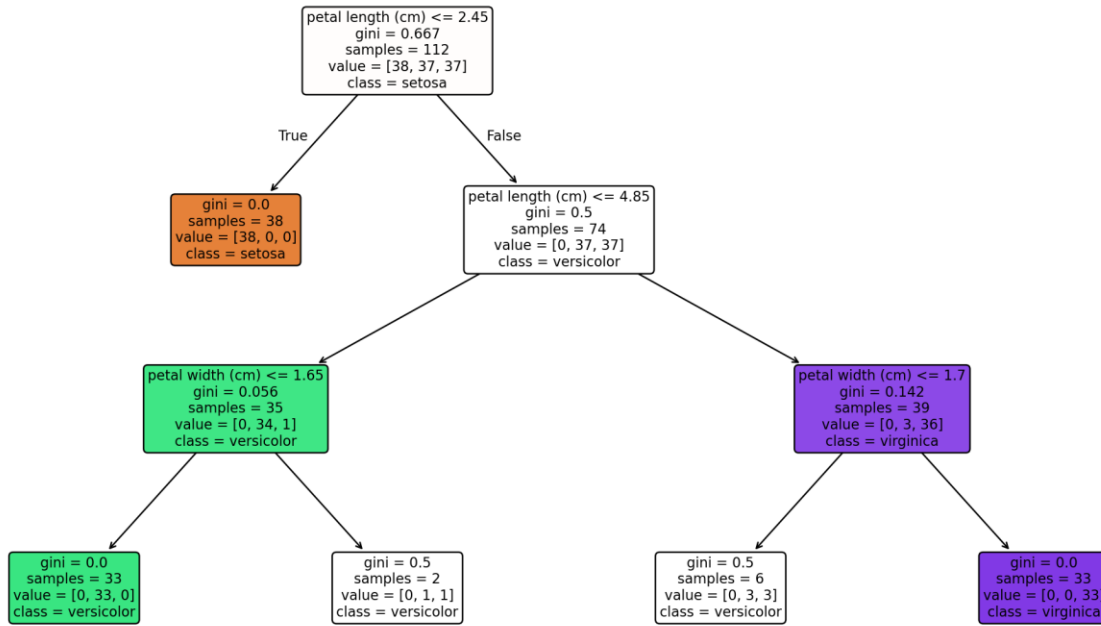
Verified sample output

```

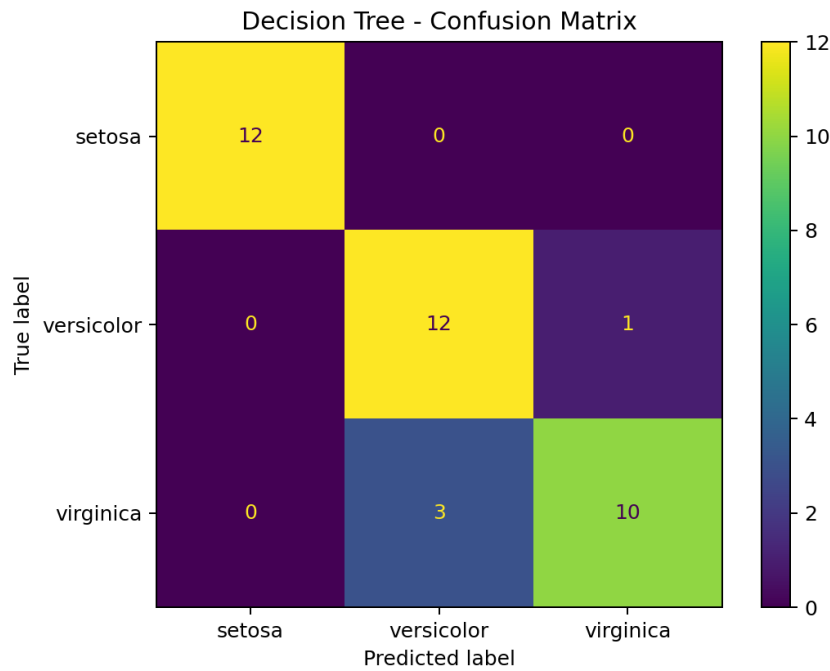
Accuracy: 0.895
Confusion Matrix:
[12, 0, 0]
[0, 12, 1]
[0, 3, 10]
Tree depth: 3
Number of leaves: 5
New flower prediction: setosa

```

Decision Tree Trained on the Iris Dataset



Verified trained Decision Tree



Confusion matrix on the test data

Interpretation

- Each path from root to leaf acts like an IF-THEN rule.
- A confusion-matrix diagonal cell represents correct predictions.
- Limiting max_depth makes the tree easier to read and can reduce overfitting.
- Feature thresholds are learned from the training data; they are not manually written.

Result: The Decision Tree classifier was trained, tested and visualized successfully. It predicted Iris species with approximately 89.5% accuracy in the verified run.

Common Errors and Solutions

Problem	Likely cause	Solution
Found input variables with inconsistent numbers of samples	X and y have different row counts	Check shapes using <code>print(X.shape, y.shape)</code> .
Tree image is too small	Default figure size is small	Use <code>plt.figure(figsize=(12, 7))</code> .
Very high training accuracy but poor testing accuracy	Tree is overfitting	Reduce <code>max_depth</code> or increase <code>min_samples_leaf</code> .

Viva-Voce Questions

1. What is the difference between a root node and a leaf node?
2. What does Gini impurity measure?
3. Why can an unrestricted Decision Tree overfit?
4. What is the purpose of `random_state`?
5. How is a confusion matrix interpreted?
6. Can a Decision Tree perform regression?

Practice / Extension Exercises

1. Change criterion to "entropy" and compare the accuracy and tree structure.
2. Remove `max_depth` and observe the new depth and number of leaves.
3. Print `model.feature_importances_` and identify the most important feature.

EXPERIMENT 2

Linear Regression

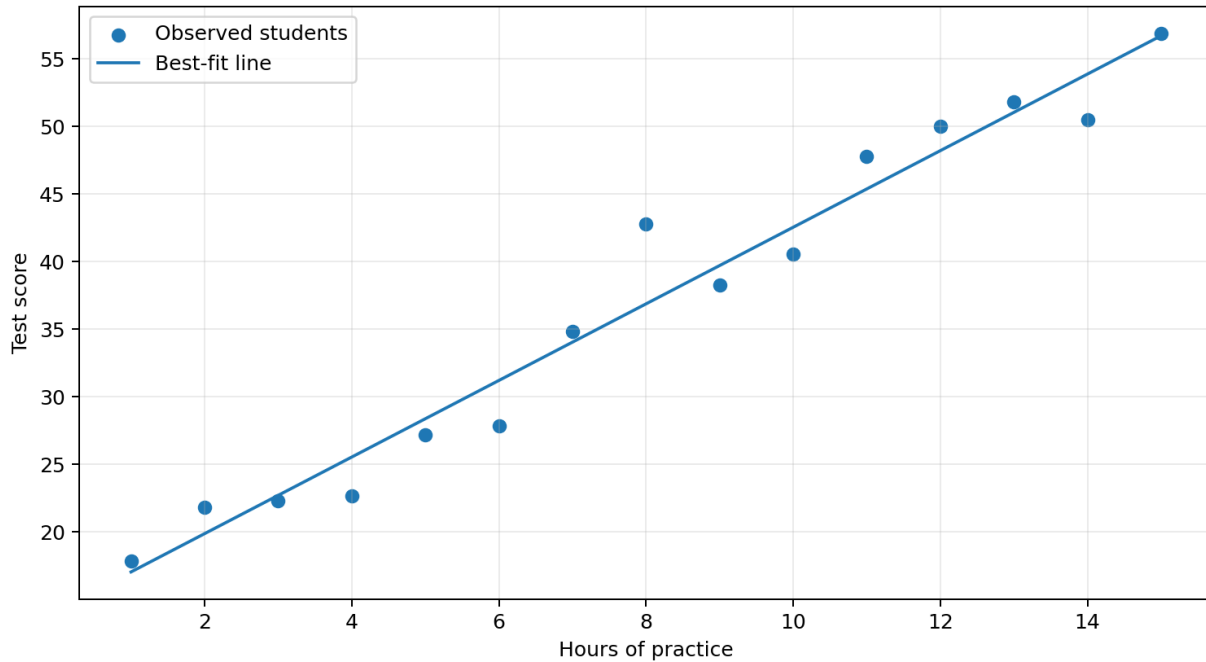
Aim: To implement simple Linear Regression for predicting a student score from study hours and evaluate the regression line.

Requirements: Python, NumPy, Matplotlib and scikit-learn.

Concept

Linear Regression predicts a continuous numerical value by fitting a straight line that minimizes squared residuals. A residual is the difference between an observed value and the model prediction.

Linear Regression: Learning a Line from Data



Linear Regression fits a line that represents the average linear trend

Model equation: $\hat{y} = b_0 + b_1 \cdot x$. Here b_0 is the intercept and b_1 is the coefficient or slope.

Metric	Meaning	Preferred value
MAE	Average absolute prediction error	Closer to 0
MSE	Average squared prediction error	Closer to 0
RMSE	Square root of MSE; same unit as target	Closer to 0
R-squared	Proportion of target variation explained	Closer to 1

Algorithm

1. Create or load paired input-output observations.
2. Reshape the input feature into a two-dimensional feature matrix.
3. Split observations into training and testing sets.
4. Fit LinearRegression on training data.
5. Read the coefficient and intercept.
6. Predict scores for the test data and a new study-hour value.
7. Evaluate MAE, MSE, RMSE and R-squared.
8. Plot observed data and the fitted regression line.

Complete Python program

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score,
```



```

)

# Study hours (X) and examination score (y)
X = np.array([
    1, 2, 2.5, 3, 4, 5,
    5.5, 6, 7, 8, 9, 10,
], dtype=float).reshape(-1, 1)

y = np.array([
    18, 25, 30, 34, 42, 52,
    55, 61, 68, 76, 86, 92,
], dtype=float)

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.25,
    random_state=42,
)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
print("Coefficient:", model.coef_[0])
print("Intercept:", model.intercept_)
print("MAE:", mean_absolute_error(y_test, y_pred))
print("MSE:", mse)
print("RMSE:", np.sqrt(mse))
print("R-squared:", r2_score(y_test, y_pred))
print("Score for 7.5 hours:", model.predict([[7.5]])[0])

# Plot full dataset and regression line
x_line = np.linspace(X.min(), X.max(), 100).reshape(-1, 1)
plt.scatter(X.ravel(), y, label="Observed data")
plt.plot(x_line.ravel(), model.predict(x_line), label="Best-fit line")
plt.xlabel("Study hours")
plt.ylabel("Score")
plt.title("Linear Regression")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

```

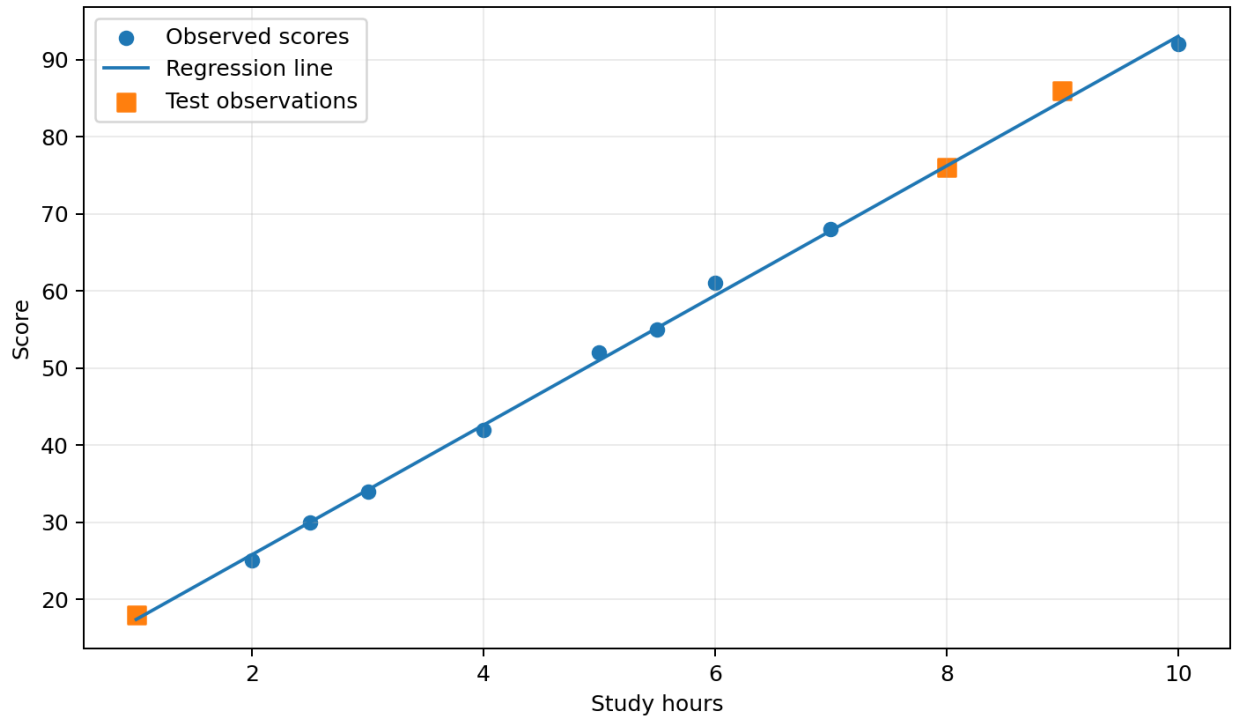
Verified sample output

```

Coefficient (slope): 8.406
Intercept: 8.970
MAE: 0.739
MSE: 0.777
RMSE: 0.881
R-squared: 0.9991
Predicted score for 7.5 hours: 72.01

```

Linear Regression: Study Hours vs Score

*Observed points and fitted regression line*

Interpretation

- The slope 8.41 means the model associates one additional study hour with an average score increase of about 8.41 points.
- The intercept is the predicted score when study hours equal zero; it may be mathematically useful even when zero lies outside the practical range.
- A high R-squared in this small constructed dataset indicates a strong linear relationship, not proof of causation.
- Extrapolating far outside the observed range can produce unrealistic predictions.

Result: Linear Regression was implemented successfully and used to predict a continuous examination score. The verified test R-squared was approximately 0.999.

Common Errors and Solutions

Problem	Likely cause	Solution
Expected 2D array, got 1D array	X was not reshaped	Use <code>X.reshape(-1, 1)</code> .
Negative R-squared	Model performs worse than predicting the test mean	Check split, outliers and whether the relationship is linear.
Plot line is zig-zag	X values used for plotting are unsorted	Create a sorted <code>x_line</code> with <code>np.linspace</code> .

Viva-Voce Questions

1. What is the difference between a coefficient and an intercept?
2. What is a residual?
3. Why does MSE penalize large errors more strongly than MAE?
4. What does R-squared equal to 1 indicate?
5. Can Linear Regression be used for class labels?
6. Why should extreme extrapolation be avoided?

Practice / Extension Exercises

1. Add an attendance feature and implement multiple Linear Regression.
2. Introduce one outlier score and observe its effect on the regression line.
3. Calculate residuals for every test observation and plot them.

EXPERIMENT 3

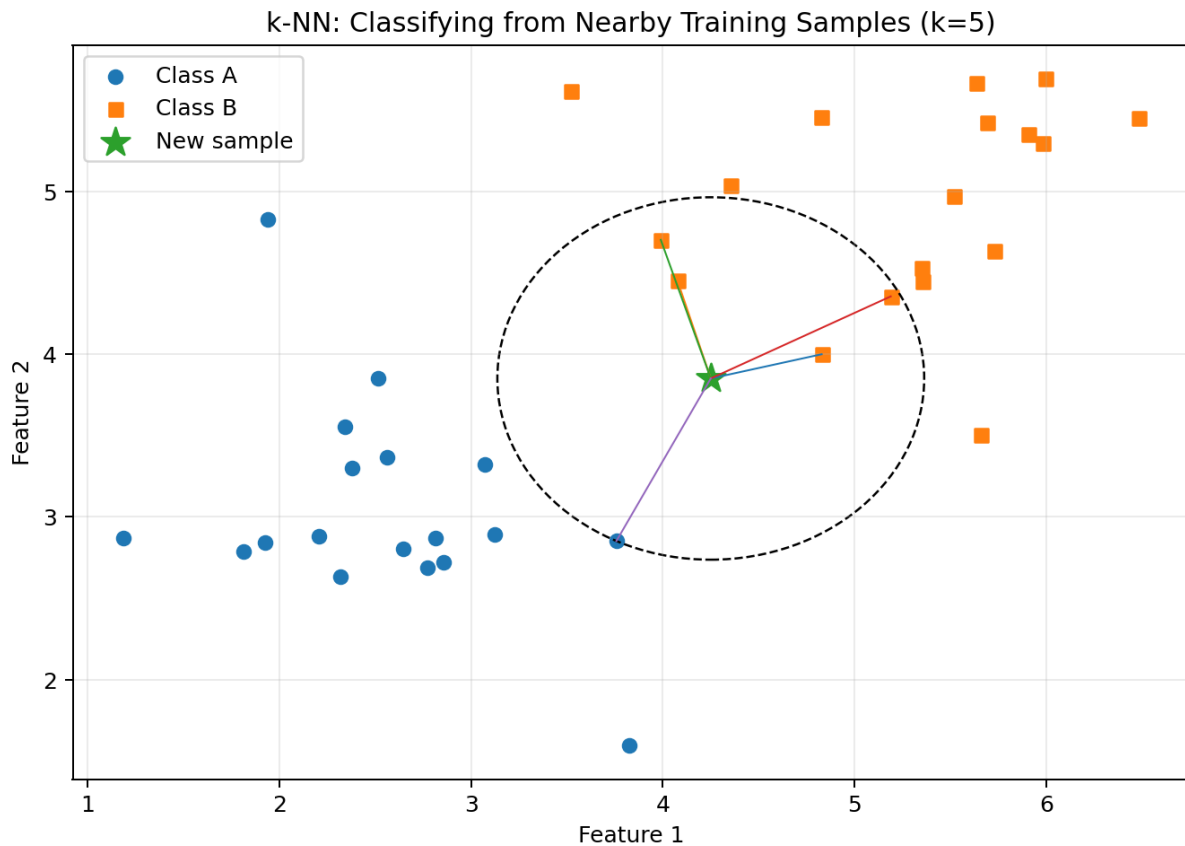
k-Nearest Neighbours (k-NN)

Aim: To implement k-NN classification on the Iris dataset using feature scaling and evaluate different values of k.

Requirements: Python, scikit-learn and Matplotlib.

Concept

k-NN is a distance-based, instance-based algorithm. It does not build a complex equation during training. For a new point, it finds the k closest labelled training points and predicts the majority class. Because distances depend on feature magnitude, scaling is generally important.



k-NN predicts from the labels of nearby training points

Euclidean distance: $d(x, z) = \sqrt{\sum((x_i - z_i)^2)}$. In scikit-learn, `metric="minkowski"` with `p=2` gives Euclidean distance.

Parameter	Effect
<code>n_neighbors (k)</code>	Number of neighbours that vote
<code>weights="uniform"</code>	Every neighbour has equal vote

Parameter	Effect
weights="distance"	Closer neighbours receive greater weight
p=1	Manhattan distance
p=2	Euclidean distance

Algorithm

1. Load the Iris dataset and split it into training and testing data.
2. Standardize the features using StandardScaler.
3. Choose a value of k.
4. For each test point, calculate distances to training observations.
5. Select the k nearest observations.
6. Predict the class with the largest vote.
7. Calculate accuracy and confusion matrix.
8. Repeat for different k values to study model behaviour.

Complete Python program

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, confusion_matrix

iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.25,
    random_state=42,
    stratify=y,
)

# Pipeline prevents leakage: scaler is fitted on training data only
model = Pipeline([
    ("scaler", StandardScaler()),
    ("knn", KNeighborsClassifier(
        n_neighbors=5,
        metric="minkowski",
        p=2,
    )),
])

model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:
", confusion_matrix(y_test, y_pred))

new_flower = [[6.2, 2.9, 4.3, 1.3]]
new_class = model.predict(new_flower)[0]
print("Prediction:", iris.target_names[new_class])

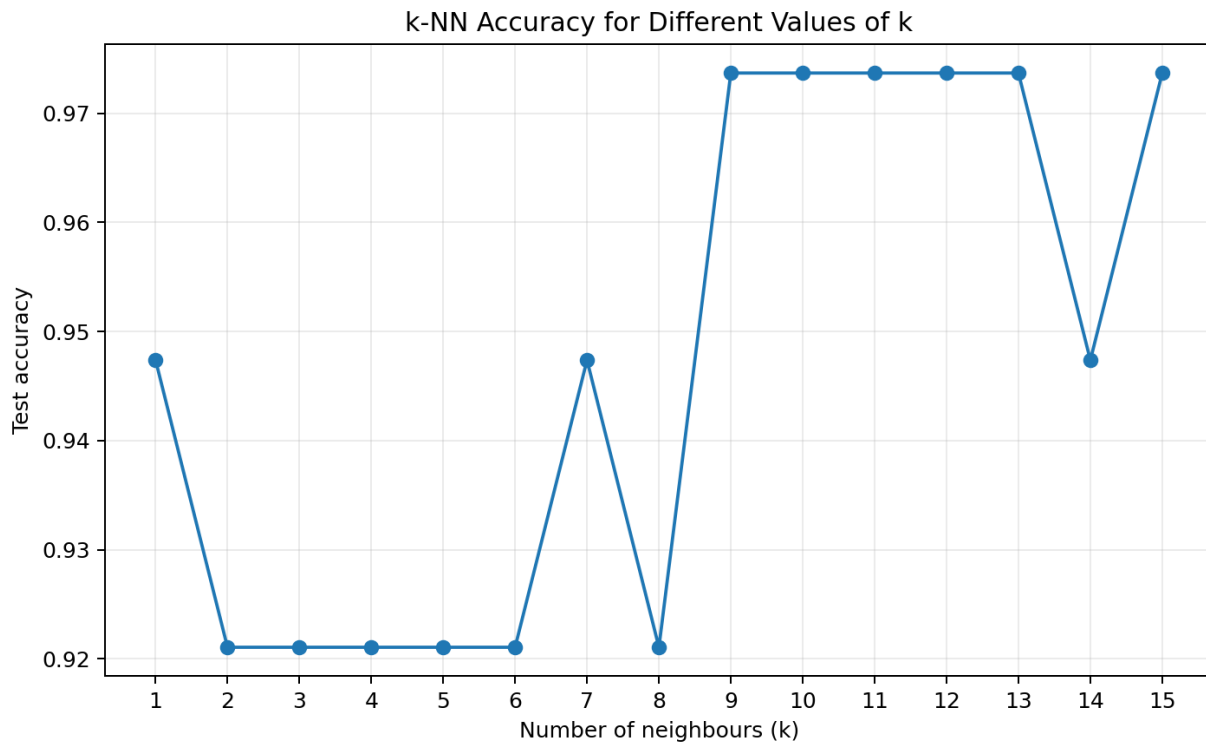
# Compare k = 1 to 15
k_values = range(1, 16)
accuracies = []
for k in k_values:
    candidate = Pipeline([
        ("scaler", StandardScaler()),
        ("knn", KNeighborsClassifier(n_neighbors=k)),
    ])
    candidate.fit(X_train, y_train)
```

```
accuracies.append(candidate.score(X_test, y_test))

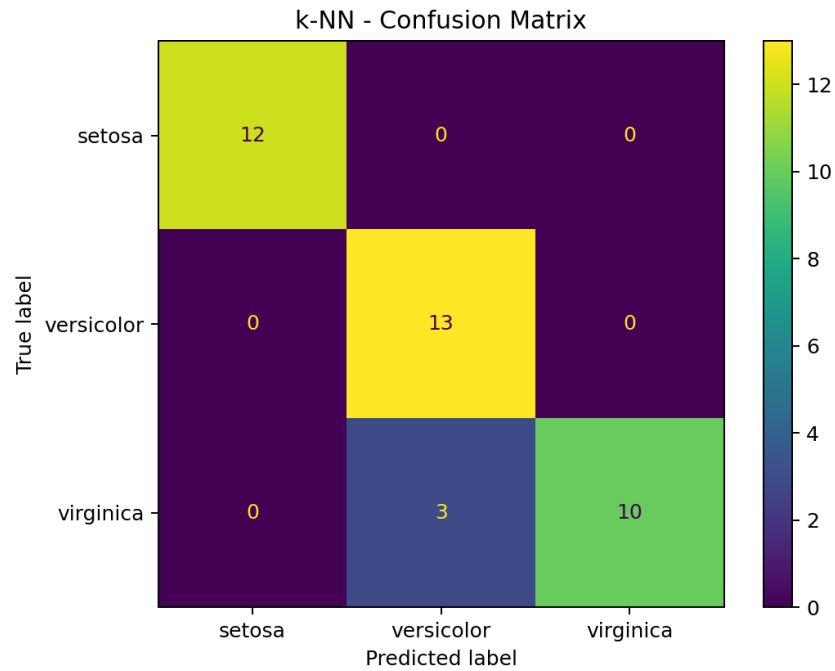
plt.plot(k_values, accuracies, marker="o")
plt.xlabel("k")
plt.ylabel("Test accuracy")
plt.title("Selecting k for k-NN")
plt.xticks(list(k_values))
plt.grid(alpha=0.3)
plt.show()
```

Verified sample output

```
Accuracy: 0.921
Confusion Matrix:
[12, 0, 0]
[0, 13, 0]
[0, 3, 10]
New flower prediction: versicolor
```



Test accuracy for different values of k

*k-NN confusion matrix*

Interpretation

- A very small k can be sensitive to noise; a very large k can oversmooth class boundaries.
- An odd k is often convenient for binary classification because it reduces ties, but it is not a universal rule.
- StandardScaler belongs inside a Pipeline so its mean and standard deviation are learned only from training data.
- k -NN prediction can become slow for very large training datasets because distances are computed at prediction time.

Result: A scaled k -NN classifier with $k=5$ was implemented successfully and achieved approximately 92.1% verified test accuracy.

Common Errors and Solutions

Problem	Likely cause	Solution
Accuracy changes greatly after adding a large-valued feature	Features are not scaled	Use StandardScaler in a Pipeline.
$n_neighbors$ is greater than number of training samples	k is too large	Select k smaller than or equal to training sample count.
Unexpectedly poor performance	Unsuitable k or noisy features	Compare several k values and inspect data quality.

Viva-Voce Questions

1. Why is k -NN called a lazy learner?
2. Why should features usually be scaled?
3. What happens when k is too small?
4. What happens when k is too large?
5. How does distance weighting differ from uniform voting?
6. Can k -NN perform regression?

Practice / Extension Exercises

1. Compare $p=1$ and $p=2$ and report which distance works better.
2. Set `weights="distance"` and compare the confusion matrix.
3. Use cross-validation to select k rather than selecting it from one test split.

EXPERIMENT 4

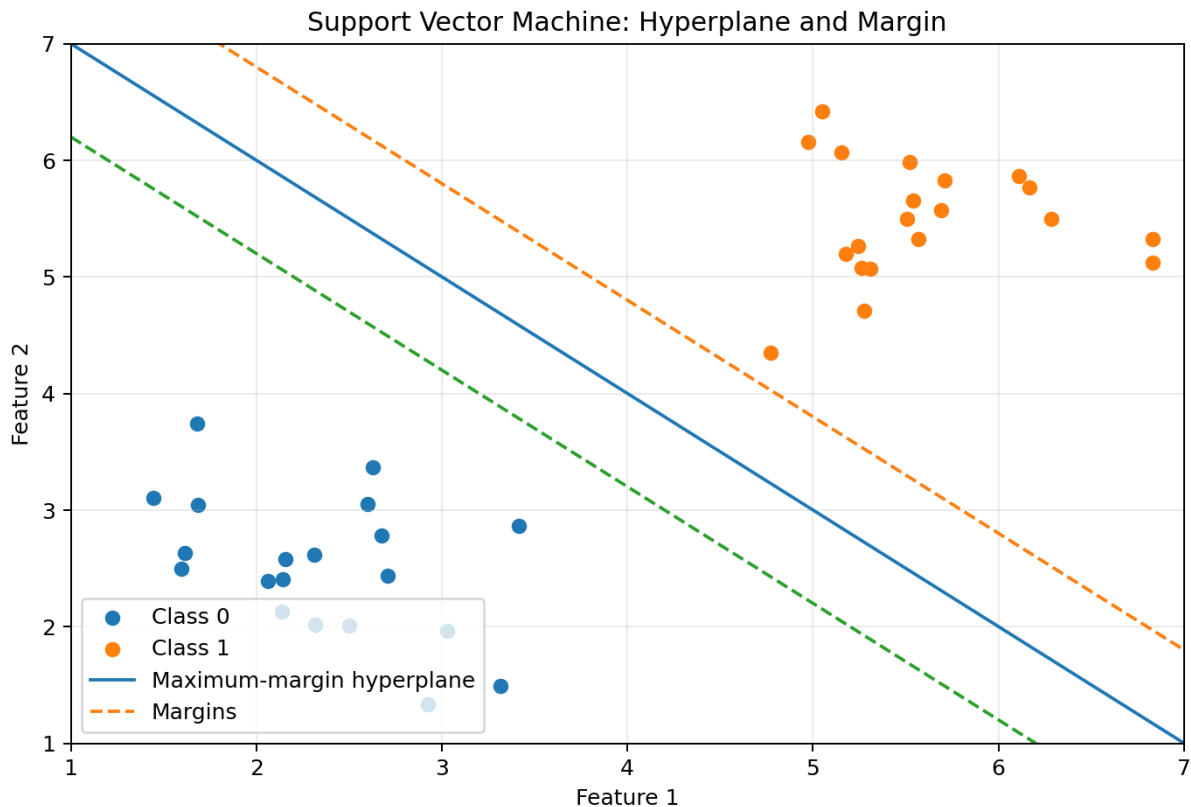
Support Vector Machine (SVM)

Aim: To implement an SVM classifier with an RBF kernel and visualize its decision regions.

Requirements: Python, scikit-learn, NumPy and Matplotlib.

Concept

An SVM searches for a decision boundary that separates classes with the widest possible margin. The training points closest to the boundary are support vectors. A kernel allows SVM to create nonlinear boundaries without explicitly constructing every transformed feature.



SVM chooses a separating boundary with a large margin

Parameter	Meaning
kernel	Shape of the decision function: linear, polynomial, RBF or sigmoid
C	Penalty for classification errors; larger C usually fits training data more strictly
gamma	Influence radius of individual samples for RBF and polynomial kernels
support vectors	Training points that define the margin and boundary

Scaling matters: SVM optimization and kernels are influenced by feature scale. Use StandardScaler before SVC for most numerical datasets.

Algorithm

1. Load and split the labelled dataset.
2. Standardize the features.
3. Choose a kernel and values of C and gamma.
4. Fit SVC on the training data.
5. Predict test labels.
6. Evaluate accuracy, confusion matrix and classification report.
7. For visualization, train another model on two selected features and draw decision regions.

Complete Python program

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, confusion_matrix

iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.25,
    random_state=42,
    stratify=y,
)

model = Pipeline([
    ("scaler", StandardScaler()),
    ("svm", SVC(
        kernel="rbf",
        C=1.0,
        gamma="scale",
    )),
])

model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:
", confusion_matrix(y_test, y_pred))
print("New prediction:", iris.target_names[
    model.predict([[6.3, 3.3, 6.0, 2.5]])[0]
])

# Optional 2D visualization using petal features only
X_2d = iris.data[:, 2:4]
visual_model = Pipeline([
    ("scaler", StandardScaler()),
    ("svm", SVC(kernel="rbf", C=1.0, gamma="scale")),
])
visual_model.fit(X_2d, y)

x_min, x_max = X_2d[:, 0].min() - 0.5, X_2d[:, 0].max() + 0.5
y_min, y_max = X_2d[:, 1].min() - 0.5, X_2d[:, 1].max() + 0.5
xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 350),
    np.linspace(y_min, y_max, 350),
)

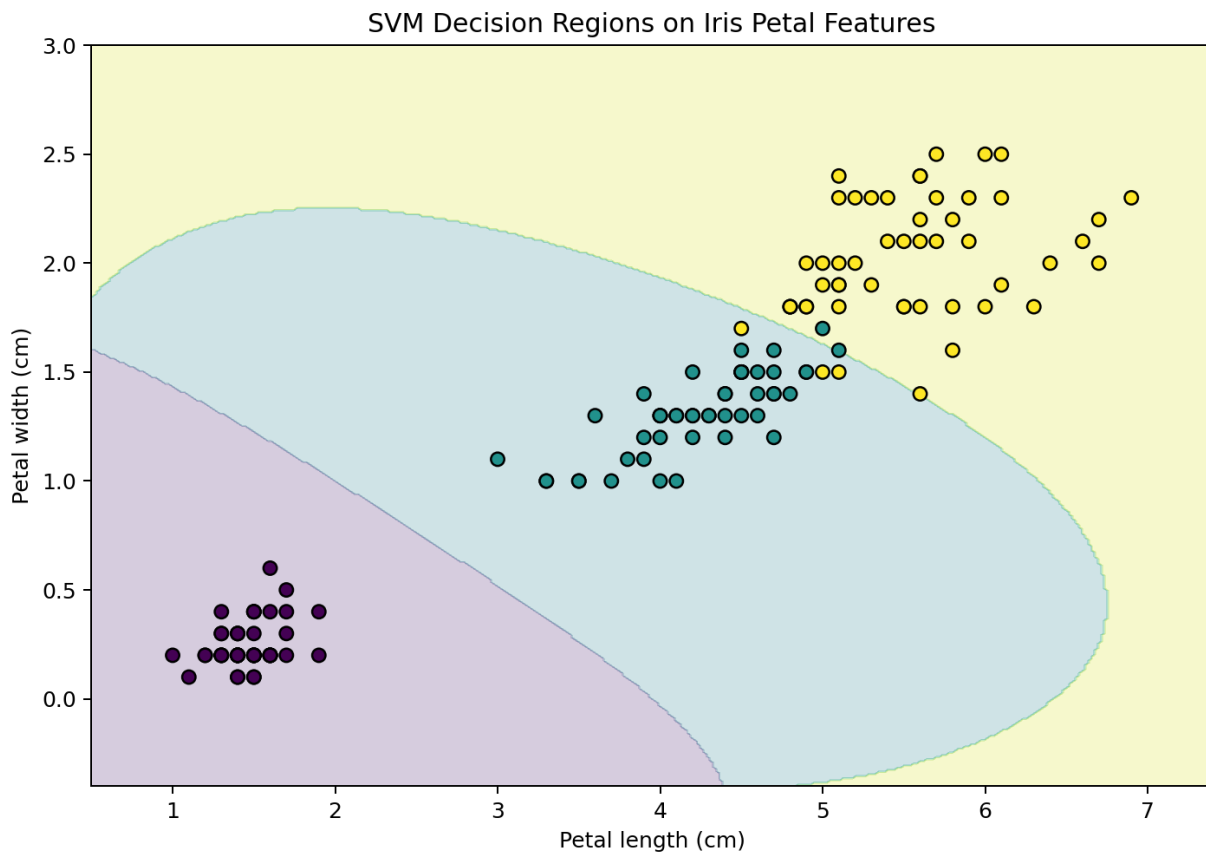
Z = visual_model.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)
```



```
plt.contourf(xx, yy, Z, alpha=0.25)
plt.scatter(X_2d[:, 0], X_2d[:, 1], c=y, edgecolor="k")
plt.xlabel("Petal length")
plt.ylabel("Petal width")
plt.title("SVM Decision Regions")
plt.show()
```

Verified sample output

```
Accuracy: 0.947
Confusion Matrix:
[12, 0, 0]
[0, 12, 1]
[0, 1, 12]
New flower prediction: virginica
```



Nonlinear decision regions created by an RBF SVM

Interpretation

- The coloured regions show the class predicted for any petal-length and petal-width combination.
- The visualization uses two features, while the evaluated model uses all four Iris features.
- Increasing C can reduce training errors but may reduce the margin and increase overfitting.
- A larger gamma makes the influence of each point more local and may produce a more irregular boundary.

Result: The RBF-kernel SVM was trained successfully and achieved approximately 94.7% verified test accuracy.

Common Errors and Solutions

Problem	Likely cause	Solution
Model is very slow on a very large dataset	Kernel SVC scales poorly with sample count	Try LinearSVC, SGDClassifier or a smaller

Problem	Likely cause	Solution
		representative dataset.
Poor accuracy despite clear pattern	Features have different scales	Standardize features inside a Pipeline.
Boundary is overly complex	C or gamma is too high	Tune C and gamma using cross-validation.

Viva-Voce Questions

1. What is a support vector?
2. What is the margin in SVM?
3. Why are kernels used?
4. What is the role of C?
5. What is the role of gamma in an RBF kernel?
6. Why should SVM inputs normally be scaled?

Practice / Extension Exercises

1. Train a linear-kernel SVM and compare it with the RBF model.
2. Try C values 0.1, 1, 10 and 100 and record the accuracy.
3. Use GridSearchCV to tune C and gamma.

EXPERIMENT 5

K-Means Clustering

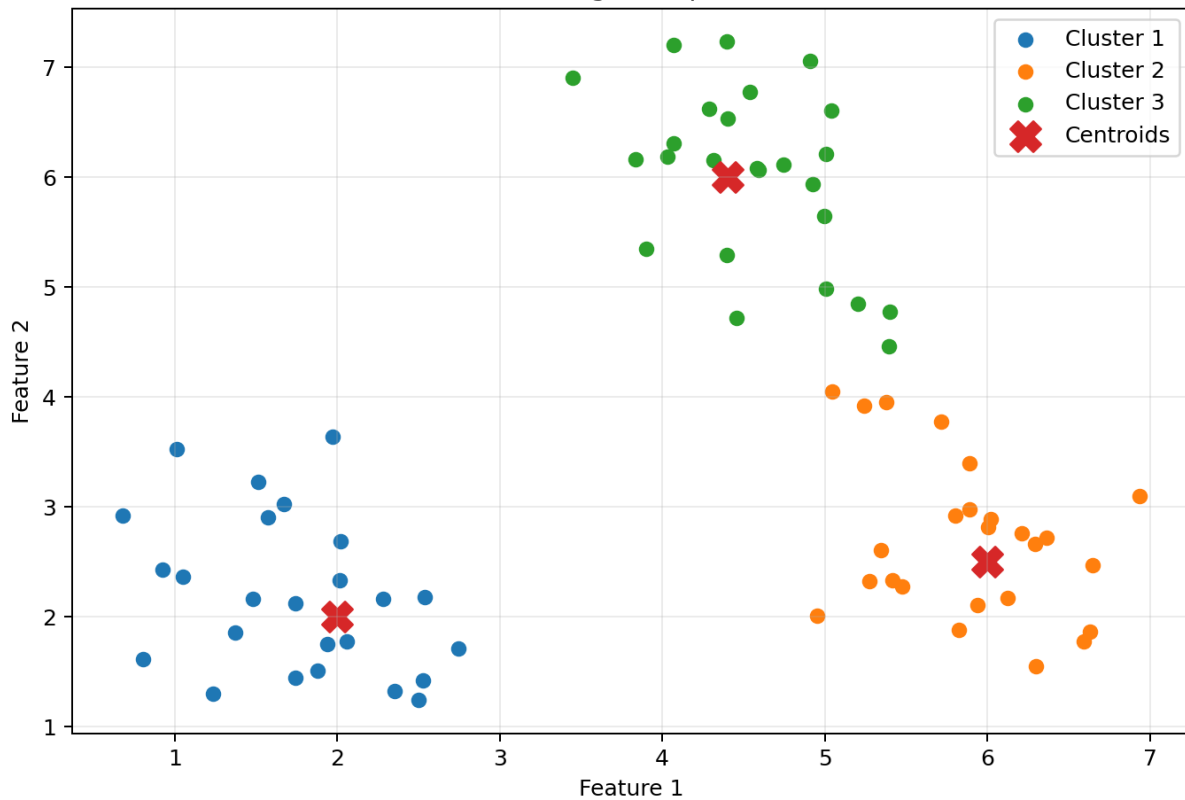
Aim: To implement K-Means clustering, select k using the elbow method and visualize the resulting clusters.

Requirements: Python, scikit-learn, NumPy and Matplotlib.

Concept

K-Means is an unsupervised algorithm that divides numeric observations into k clusters. Every observation belongs to the nearest centroid, and each centroid is repeatedly updated to the mean of its assigned observations. The algorithm minimizes within-cluster squared distances, reported as inertia.

K-means Clustering: Groups and Centroids



K-Means alternates between assigning points and updating centroids

Objective: Minimize the sum of squared distances from every point to its assigned cluster centroid.

Measure	Interpretation
Inertia	Within-cluster squared distance; lower for the same dataset, but always decreases as k grows
Elbow method	Looks for a point after which adding clusters gives smaller improvement
Silhouette score	Compares within-cluster cohesion with separation; ranges approximately from -1 to 1
Adjusted Rand Index	Compares clusters to known labels when labels are available for evaluation only

Algorithm

1. Choose k initial centroids.
2. Assign every observation to the nearest centroid.
3. Recalculate each centroid as the mean of its assigned observations.
4. Repeat assignment and update steps until centroids stabilize or an iteration limit is reached.
5. Evaluate inertia and silhouette score.
6. Repeat for several k values and inspect the elbow curve.
7. Reduce to two dimensions for visualization if the original data has many features.

Complete Python program

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score

iris = load_iris()
X = iris.data

# Scaling gives features comparable influence
X_scaled = StandardScaler().fit_transform(X)

model = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10,
)
labels = model.fit_predict(X_scaled)

print("Cluster labels (first 20):", labels[:20])
print("Inertia:", model.inertia_)
print("Silhouette score:", silhouette_score(X_scaled, labels))

# Elbow method
inertias = []
for k in range(1, 8):
    candidate = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10,
    )
    candidate.fit(X_scaled)
    inertias.append(candidate.inertia_)

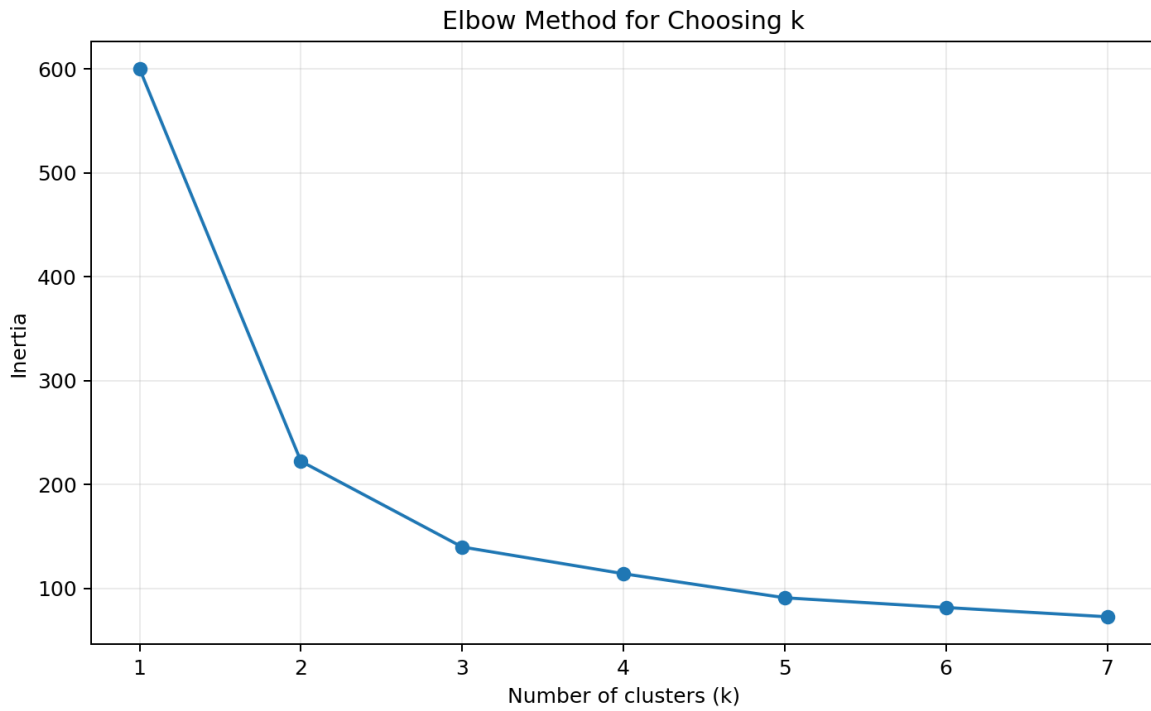
plt.plot(range(1, 8), inertias, marker="o")
plt.xlabel("Number of clusters (k)")
plt.ylabel("Inertia")
plt.title("Elbow Method")
plt.grid(alpha=0.3)
plt.show()

# Visualize clusters after PCA reduction to 2D
pca = PCA(n_components=2)
X_2d = pca.fit_transform(X_scaled)
centers_2d = pca.transform(model.cluster_centers_)

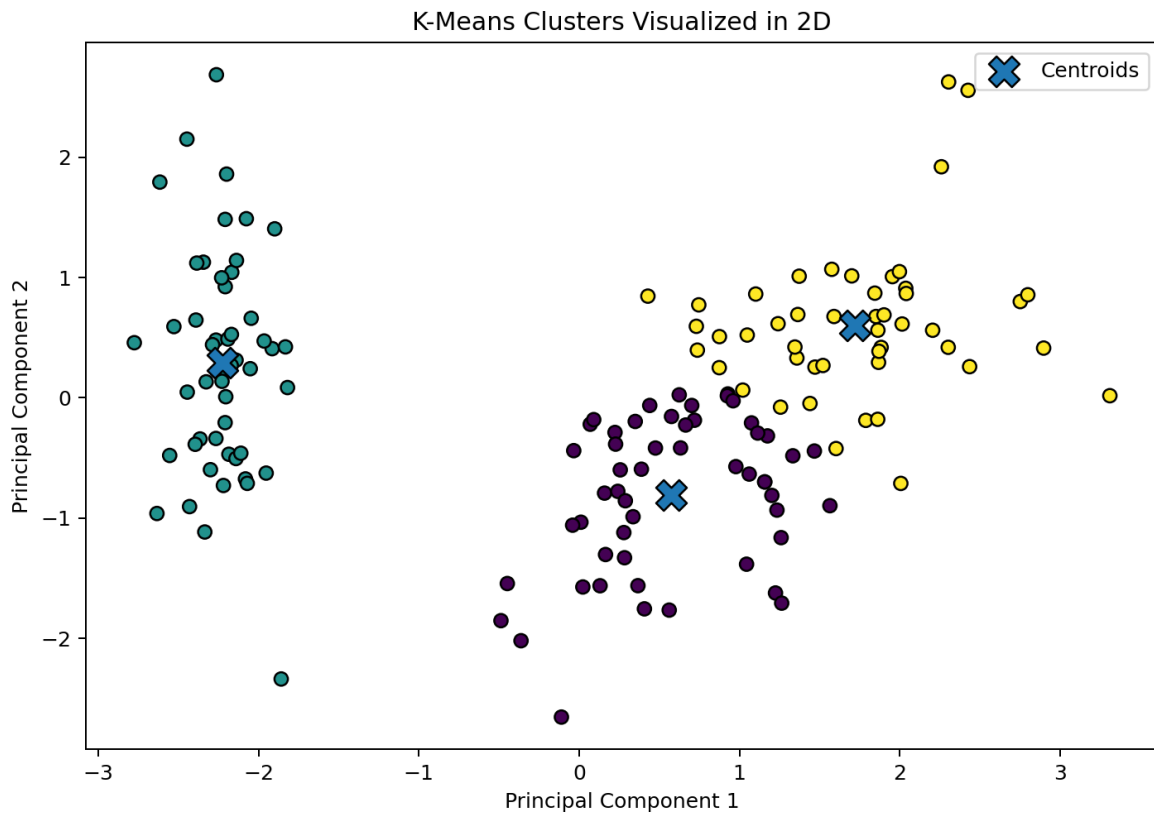
plt.scatter(X_2d[:, 0], X_2d[:, 1], c=labels, edgecolor="k")
plt.scatter(
    centers_2d[:, 0], centers_2d[:, 1],
    marker="X", s=220, edgecolor="k",
    label="Centroids",
)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("K-Means Clusters")
plt.legend()
plt.show()
```

Verified sample output

```
Inertia: 139.820
Silhouette score: 0.460
Cluster sizes: [53, 50, 47]
Adjusted Rand Index against Iris labels: 0.620
```



Elbow curve for k = 1 to 7



Three K-Means clusters projected into two PCA dimensions

Interpretation

- Cluster numbers 0, 1 and 2 are arbitrary identifiers; they are not equal to original species numbers.
- The elbow method is a visual heuristic, not a guaranteed mathematical answer.

- K-Means works best when clusters are compact and roughly spherical in the chosen feature space.
- Scaling prevents large-unit features from dominating Euclidean distances.

Result: K-Means successfully divided the standardized Iris observations into three clusters. The verified silhouette score was approximately 0.460.

Common Errors and Solutions

Problem	Likely cause	Solution
Different cluster labels on repeated runs	Random initialization	Set <code>random_state</code> and a suitable <code>n_init</code> .
One feature dominates clustering	Features use different scales	Standardize the features.
Poor clusters for curved or unequal-density data	K-Means assumptions do not match data	Consider DBSCAN, Mean Shift or spectral clustering.

Viva-Voce Questions

1. Why is K-Means an unsupervised algorithm?
2. What is a centroid?
3. What does inertia measure?
4. Why does inertia always decrease as k increases?
5. What does a high silhouette score indicate?
6. Why are cluster labels arbitrary?

Practice / Extension Exercises

1. Run K-Means with $k=2, 3, 4$ and 5 and compare silhouette scores.
2. Run the algorithm without `StandardScaler` and compare the clusters.
3. Apply K-Means to a customer dataset containing annual income and spending score.

EXPERIMENT 6

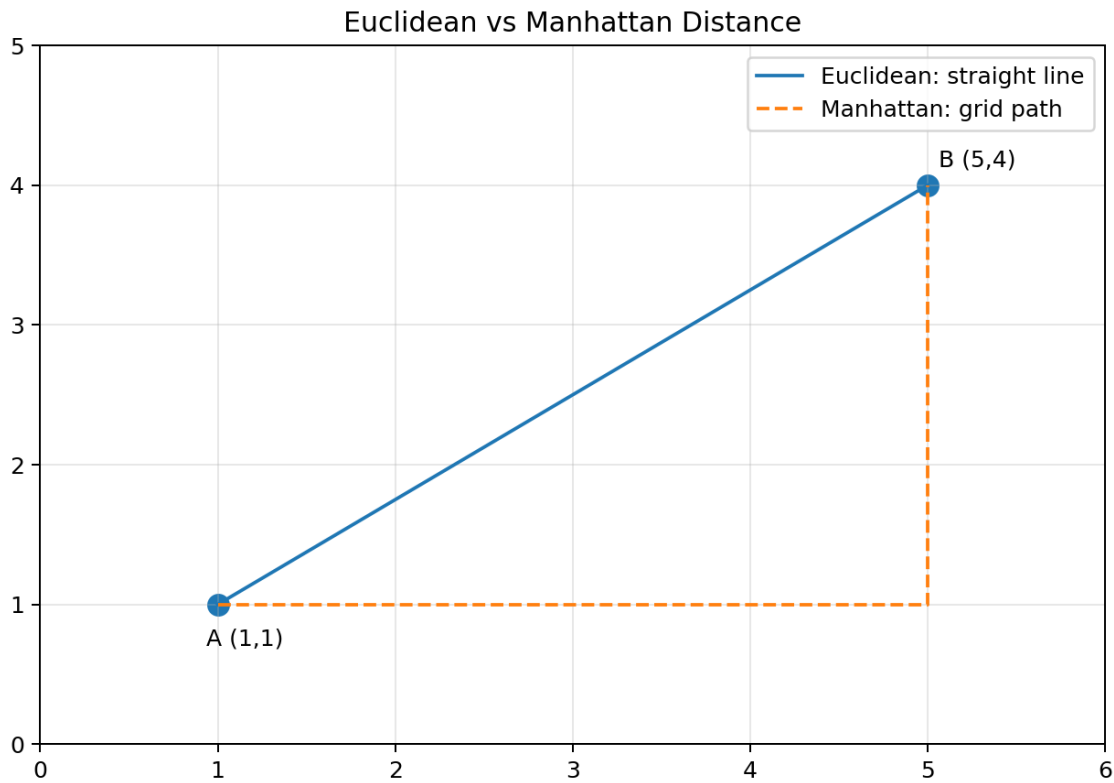
Various Distance Metrics

Aim: To calculate and compare Euclidean, Manhattan, Minkowski, Chebyshev, Cosine, Hamming, Jaccard and Mahalanobis distances using SciPy.

Requirements: Python, NumPy, SciPy and Matplotlib.

Concept

A distance or dissimilarity measure quantifies how different two observations are. The correct measure depends on the data type and meaning. For example, Euclidean distance is natural for geometric numeric data, cosine distance focuses on direction, and Jaccard distance is useful for sets or binary presence/absence features.

*Geometric intuition for Euclidean and Manhattan distance*

Metric	Core idea	Typical data/use
Euclidean	Straight-line distance	Continuous numeric geometry
Manhattan / cityblock	Sum of absolute coordinate differences	Grid movement, robust alternative
Minkowski	General p-norm family	p=1 Manhattan, p=2 Euclidean
Chebyshev	Largest coordinate difference	Maximum-step movement
Cosine distance	One minus cosine similarity	Text vectors and direction-based comparison
Hamming	Proportion of positions that differ	Codes, equal-length categorical/binary sequences
Jaccard	One minus intersection-over-union	Sets and boolean presence/absence
Mahalanobis	Accounts for scale and feature correlation	Correlated multivariate numeric data

Selected Formulas

- Euclidean: $\sqrt{\sum (u_i - v_i)^2}$
- Manhattan: $\sum |u_i - v_i|$
- Minkowski: $(\sum |u_i - v_i|^p)^{1/p}$
- Chebyshev: $\max(|u_i - v_i|)$
- Cosine distance: $1 - (u \cdot v) / (\|u\| \|v\|)$
- Hamming distance in SciPy: proportion of unequal positions
- Jaccard distance for boolean vectors: $1 - \text{intersection} / \text{union}$

Algorithm

1. Create two vectors of equal length.
2. Import `scipy.spatial.distance`.

3. Call the appropriate distance function for each metric.
4. Use boolean vectors for Jaccard presence/absence comparison.
5. For Mahalanobis distance, estimate the covariance matrix and use its inverse or pseudoinverse.
6. Use cdist to calculate all pairwise distances between two collections of observations.
7. Compare results and explain why different metrics produce different values.

Complete Python program

```
import numpy as np
from scipy.spatial import distance

u = np.array([1, 2, 3, 0], dtype=float)
v = np.array([4, 0, 3, 1], dtype=float)

print("Euclidean:", distance.euclidean(u, v))
print("Manhattan:", distance.cityblock(u, v))
print("Minkowski (p=3):", distance.minkowski(u, v, p=3))
print("Chebyshev:", distance.chebyshev(u, v))
print("Cosine distance:", distance.cosine(u, v))
print("Hamming distance:", distance.hamming(u, v))

# Jaccard is commonly used with boolean vectors
b1 = np.array([1, 0, 1, 1, 0], dtype=bool)
b2 = np.array([1, 1, 0, 1, 0], dtype=bool)
print("Jaccard distance:", distance.jaccard(b1, b2))

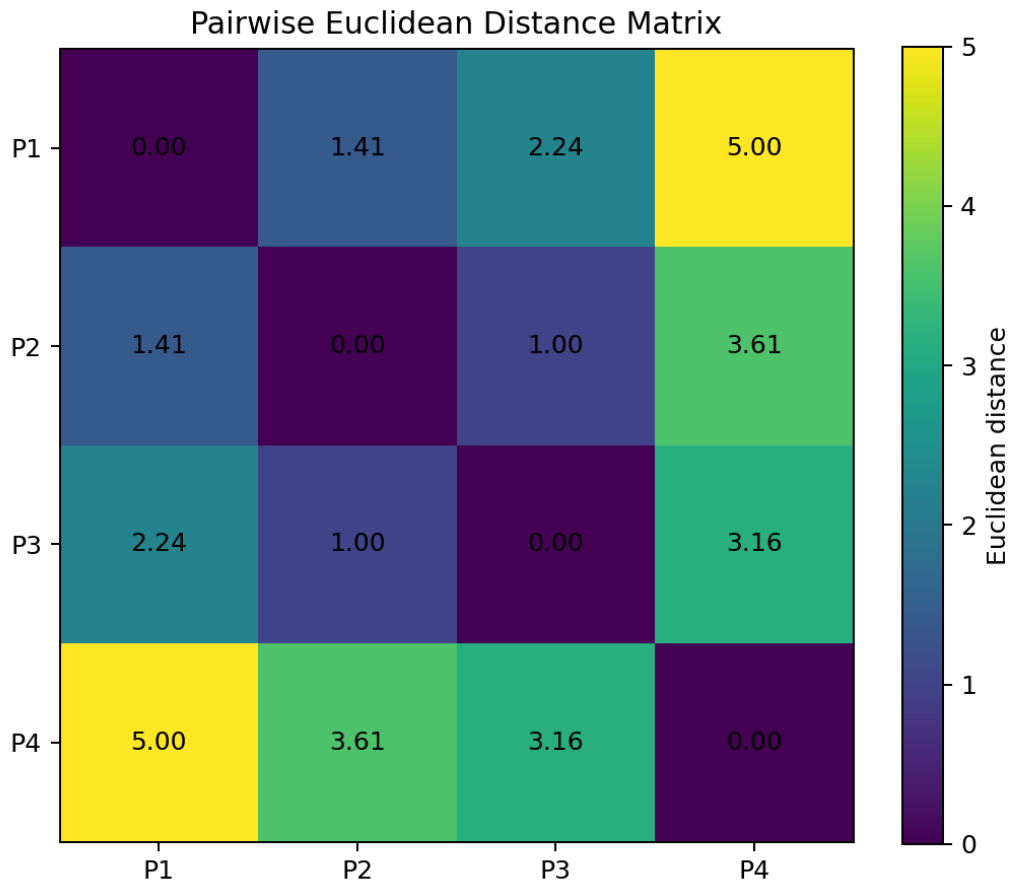
# Mahalanobis distance requires inverse covariance matrix
observations = np.array([
    [1, 2],
    [2, 4],
    [3, 5],
    [4, 7],
    [5, 8],
], dtype=float)

covariance = np.cov(observations.T)
VI = np.linalg.pinv(covariance)
mahalanobis_value = distance.mahalanobis(
    observations[0], observations[-1], VI
)
print("Mahalanobis distance:", mahalanobis_value)

# Pairwise Euclidean distance matrix
points = np.array([
    [0, 0],
    [1, 1],
    [2, 1],
    [3, 4],
], dtype=float)
D = distance.cdist(points, points, metric="euclidean")
print("Pairwise distance matrix:
", np.round(D, 3))
```

Verified sample output

```
Euclidean: 3.7417
Manhattan: 6.0000
Minkowski (p=3): 3.3019
Chebyshev: 3.0000
Cosine distance: 0.3186
Hamming distance: 0.7500
Jaccard distance: 0.5000
Mahalanobis distance: 2.5298
```

Manual Check for u and v

For $u = [1, 2, 3, 0]$ and $v = [4, 0, 3, 1]$, coordinate differences are $[-3, 2, 0, -1]$. Therefore Manhattan distance $= 3 + 2 + 0 + 1 = 6$, while Euclidean distance $= \sqrt{9 + 4 + 0 + 1} = \sqrt{14} = 3.7417$.

Interpretation

- Distance values from different metrics are not directly interchangeable because they use different geometry.
- Cosine distance can be small even when vector magnitudes differ, provided their directions are similar.
- Hamming and Jaccard have different meanings; Hamming counts positional mismatch while Jaccard ignores joint zeros in boolean data.
- Mahalanobis distance depends on the covariance structure estimated from a dataset, not only the two endpoints.

Result: Eight distance metrics and a complete pairwise distance matrix were calculated successfully using SciPy.

Common Errors and Solutions

Problem	Likely cause	Solution
ValueError: Input vector should be 1-D	A matrix was passed to a vector distance function	Use one-dimensional u and v, or use cdist for matrices.
Singular matrix for Mahalanobis	Covariance matrix cannot be inverted	Use np.linalg.pinv or remove redundant features.
Jaccard result is unexpected	Numeric values were treated as truth values	Convert meaningful presence/absence data to boolean first.

Viva-Voce Questions

1. When are Euclidean and Manhattan distances equal?
2. What values of p produce Manhattan and Euclidean distance in Minkowski distance?
3. What does cosine distance measure?
4. Why is Jaccard useful for sets?
5. What extra information does Mahalanobis distance use?
6. What is the difference between `pdist` and `cdist`?

Practice / Extension Exercises

1. Implement Euclidean and Manhattan distances manually without SciPy.
2. Calculate distances among five cities represented by two coordinates.
3. Use `cdist` with metrics "cityblock", "cosine" and "chebyshev" and compare matrices.

EXPERIMENT 7

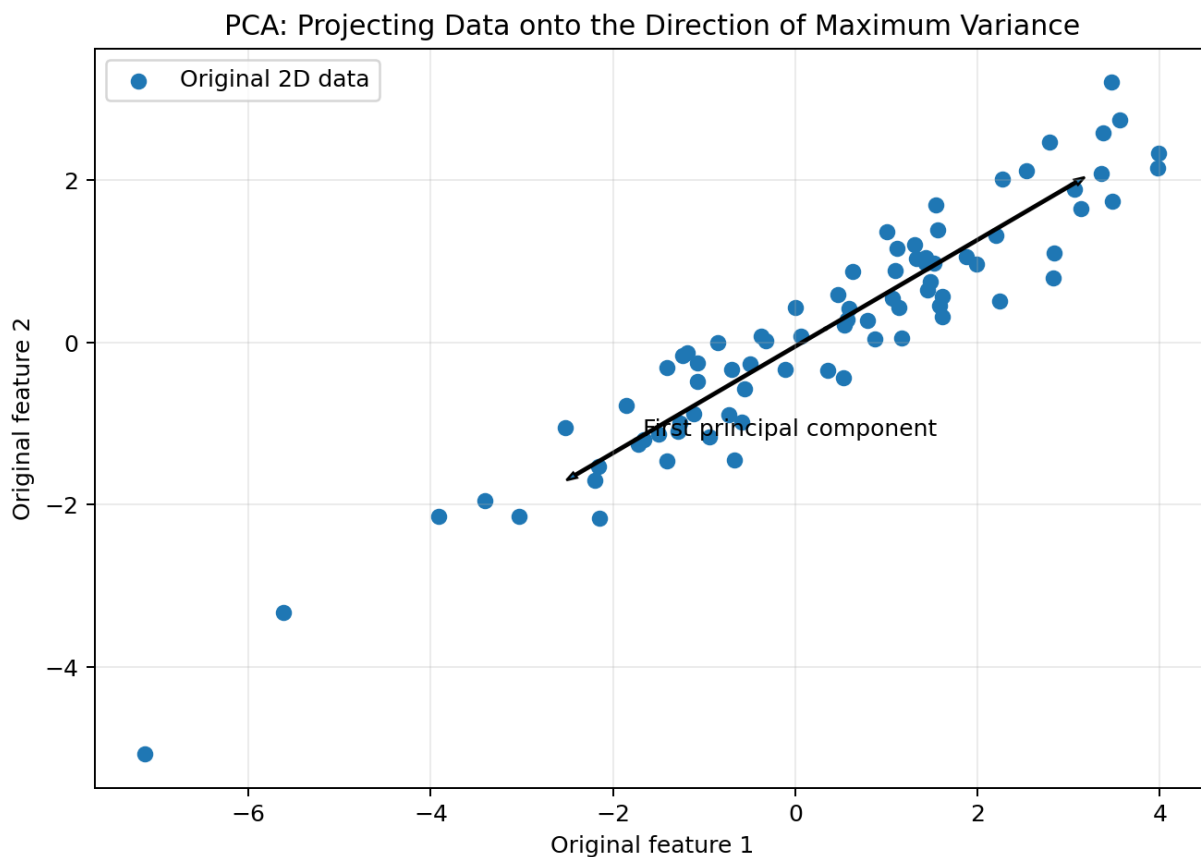
Dimensionality Reduction Techniques

Aim: To implement PCA, LDA and t-SNE for reducing the Wine dataset from 13 features to two dimensions and visualize the transformed data.

Requirements: Python, scikit-learn, NumPy and Matplotlib.

Concept

Dimensionality reduction converts many original features into fewer new dimensions while preserving useful structure. It supports visualization, compression, noise reduction and sometimes faster modelling. Different techniques preserve different kinds of information.



Technique	Uses labels?	Main purpose	Key caution
PCA	No	Linear variance-preserving compression	Components may be difficult to interpret
LDA	Yes	Linear projection that separates known classes	Maximum components are limited by number of classes
t-SNE	No	Nonlinear visualization of local neighbourhoods	Distances and cluster sizes in the plot can be misleading

PCA explained variance: `explained_variance_ratio_` reports the fraction of total standardized variance represented by each principal component.

Algorithm

1. Load the Wine dataset containing 13 numerical features.
2. Standardize all features because PCA is scale-sensitive.
3. Apply PCA with `n_components=2` and inspect explained variance.
4. Apply LDA with `n_components=2` using both `X` and class labels `y`.
5. Apply t-SNE with two output dimensions for nonlinear visualization.
6. Plot the transformed observations and colour them by known class for interpretation.
7. Compare shapes and discuss what each method preserves.

Complete Python program

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.manifold import TSNE

wine = load_wine()
X = wine.data
y = wine.target

X_scaled = StandardScaler().fit_transform(X)
print("Original shape:", X_scaled.shape)

# 1. PCA - unsupervised linear reduction
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
print("PCA shape:", X_pca.shape)
print("PCA explained variance:", pca.explained_variance_ratio_)

# 2. LDA - supervised linear reduction
lda = LinearDiscriminantAnalysis(n_components=2)
X_lda = lda.fit_transform(X_scaled, y)
print("LDA shape:", X_lda.shape)

# 3. t-SNE - nonlinear visualization
# perplexity must be smaller than the number of samples
tsne = TSNE(
    n_components=2,
    random_state=42,
    init="pca",
    learning_rate="auto",
    perplexity=30,
    max_iter=750,
)
X_tsne = tsne.fit_transform(X_scaled)
print("t-SNE shape:", X_tsne.shape)

# Create one plot at a time
for transformed, title in [
```

```

(X_pca, "PCA Projection"),
(X_lda, "LDA Projection"),
(X_tsne, "t-SNE Projection"),
]:
plt.figure(figsize=(8, 5))
plt.scatter(
    transformed[:, 0],
    transformed[:, 1],
    c=y,
    edgecolor="k",
)
plt.xlabel("Component 1")
plt.ylabel("Component 2")
plt.title(title)
plt.show()

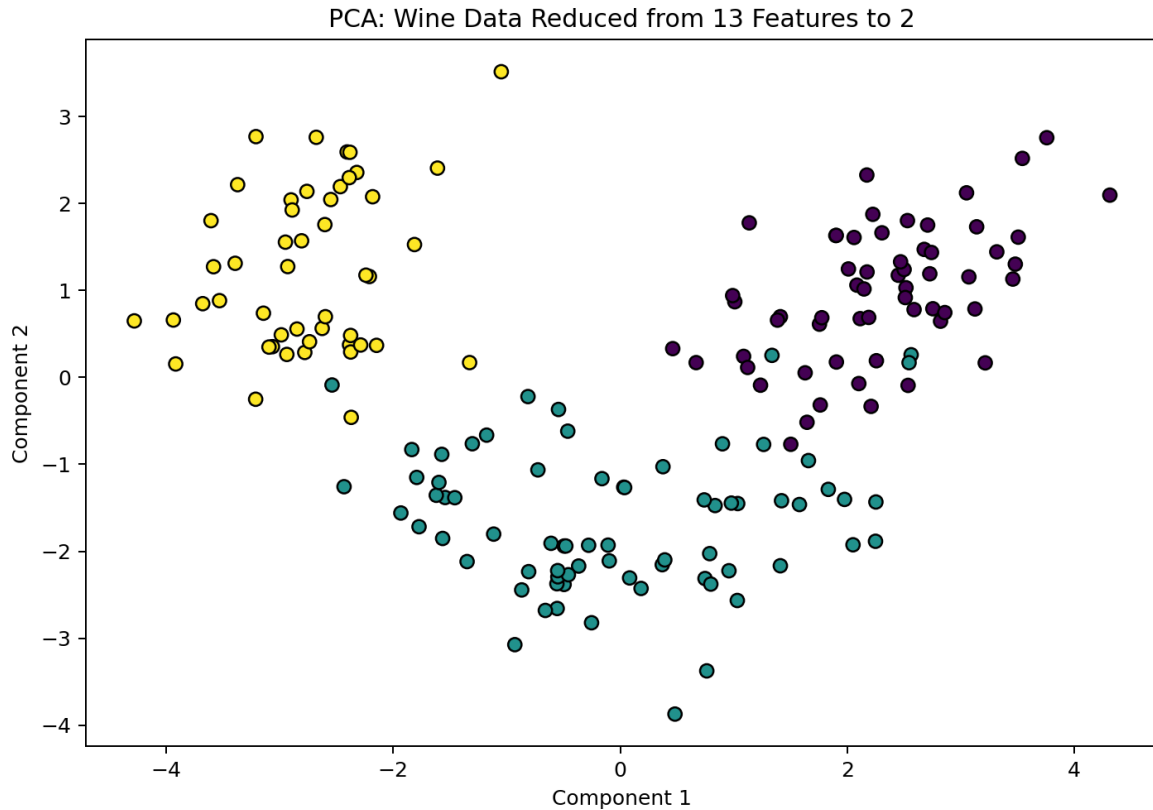
```

Verified sample output

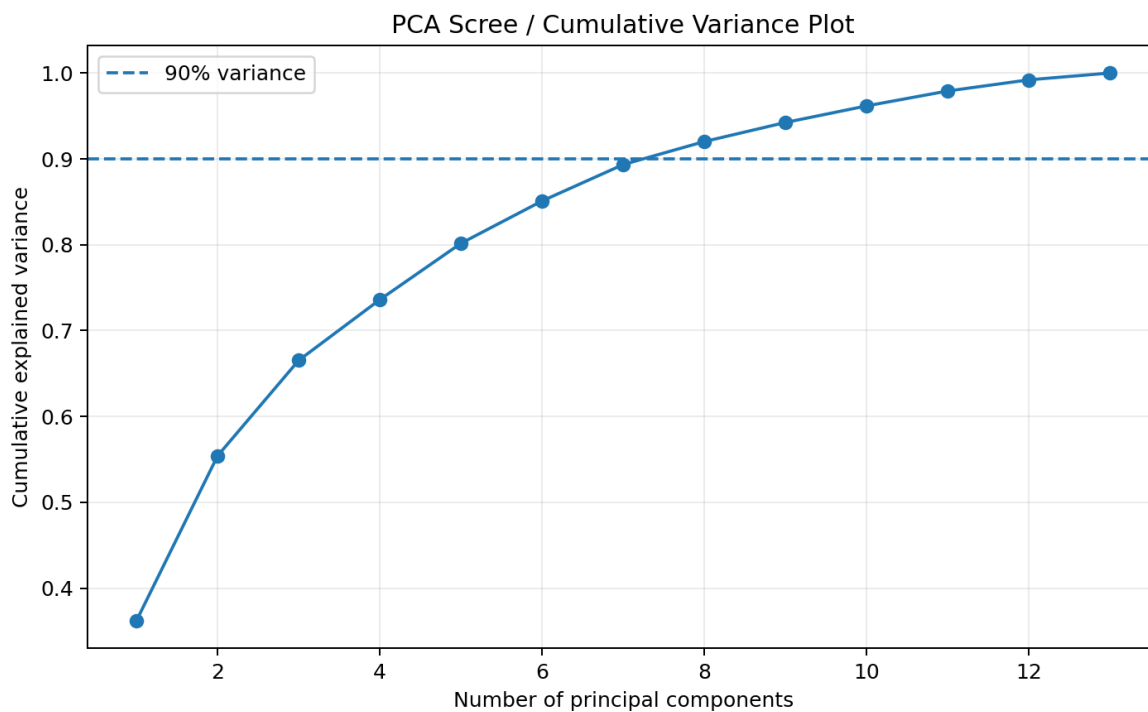
```

Original shape: (178, 13)
PCA shape: (178, 2)
PCA explained variance ratios: [0.362, 0.1921]
Total variance in first two PCA components: 0.554
LDA shape: (178, 2)
t-SNE shape: (178, 2)

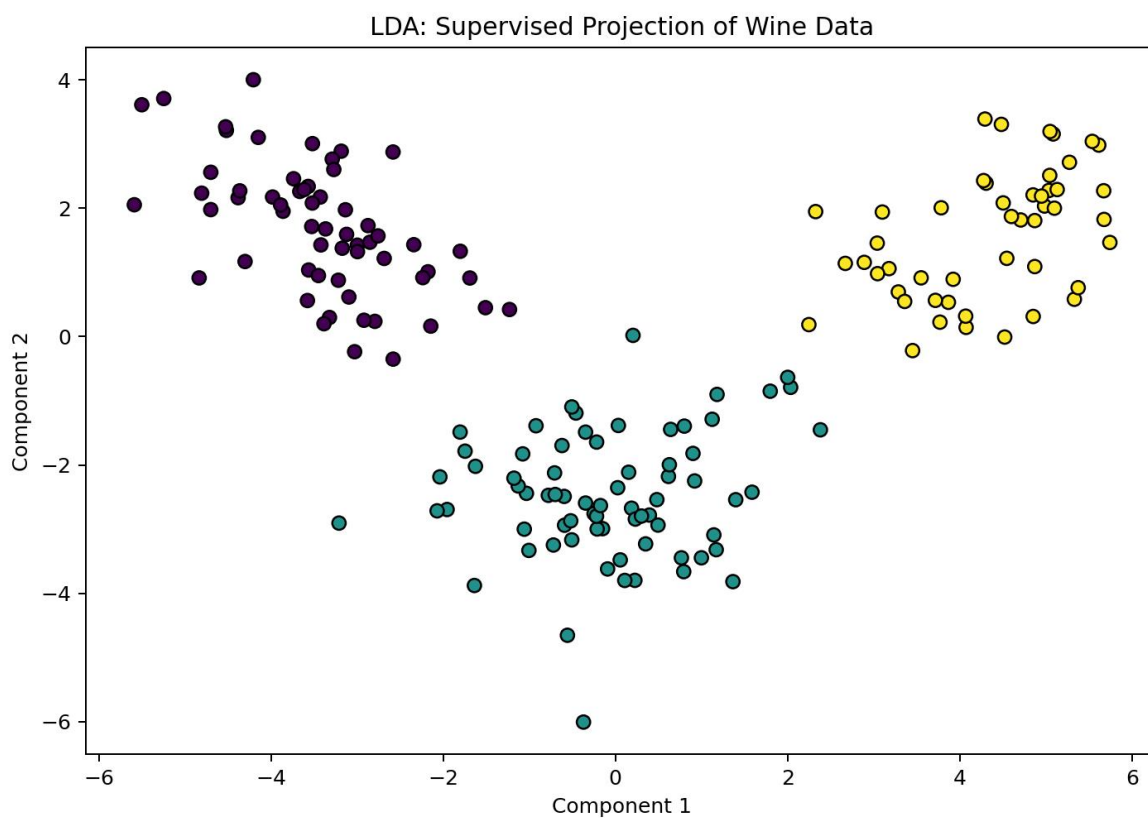
```



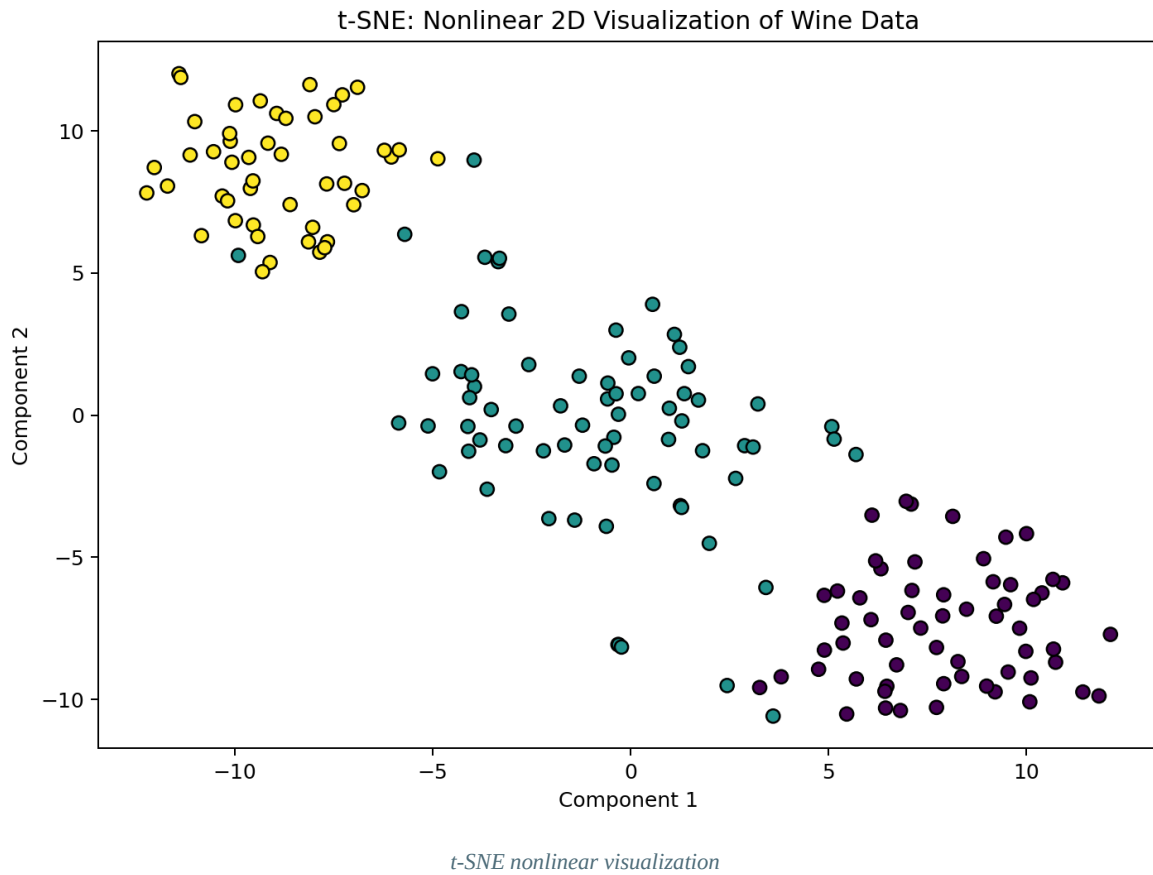
PCA projection of standardized Wine data



Cumulative explained variance for increasing PCA components



LDA projection uses class labels to improve separation



Interpretation

- The first two PCA components preserve approximately 55.4% of total standardized variance.
- LDA uses known classes and therefore often separates them more strongly than unsupervised PCA.
- t-SNE is mainly a visualization method; its axes have no direct original-feature meaning.
- A two-dimensional plot is a simplified representation and may discard useful information.

Result: PCA, LDA and t-SNE were implemented successfully, reducing 13 Wine features to two dimensions for comparison and visualization.

Common Errors and Solutions

Problem	Likely cause	Solution
perplexity must be less than n_samples	t-SNE perplexity is too large	Choose a value below the number of observations.
LDA n_components is too large	Requested components exceed min(features, classes - 1)	For three classes, use at most two LDA components.
PCA dominated by large-unit features	Data was not standardized	Apply StandardScaler before PCA when feature units differ.

Viva-Voce Questions

1. What is dimensionality reduction?
2. Is PCA supervised or unsupervised?
3. What does PCA explained variance mean?
4. Why does LDA require class labels?

5. Why is t-SNE mainly used for visualization?
6. Why should PCA inputs often be standardized?
7. Can transformed PCA components be directly interpreted as original features?

Practice / Extension Exercises

1. Find how many PCA components are needed to retain at least 90% variance.
2. Train a classifier using all Wine features and then using PCA features; compare accuracy and speed.
3. Change t-SNE perplexity to 5, 15 and 40 and compare the plots.

Practical Examination Quick Reference

Task type	Algorithm / tool	Minimum evaluation
Classification	Decision Tree, k-NN, SVM	Accuracy + confusion matrix
Regression	Linear Regression	MAE or RMSE + R-squared
Clustering	K-Means	Inertia + silhouette score
Similarity	scipy.spatial.distance	State metric and data suitability
Dimensionality reduction	PCA, LDA, t-SNE	Output shape + plot + interpretation

Model Comparison Summary

Algorithm	Learning type	Output	Scaling needed?	Strength	Main limitation
Decision Tree	Supervised	Class or number	Usually no	Readable rules	Can overfit
Linear Regression	Supervised	Continuous number	Not always, but useful in pipelines	Simple and interpretable	Assumes linear relationship
k-NN	Supervised	Class or number	Yes	Simple local reasoning	Slow prediction on large data
SVM	Supervised	Class or number	Yes	Strong margins and kernels	Parameter tuning and scaling
K-Means	Unsupervised	Cluster ID	Usually yes	Fast compact clustering	Requires k and spherical-cluster assumption
PCA	Unsupervised transform	Components	Usually yes	Compression and denoising	Linear and less interpretable
LDA	Supervised transform	Components	Usually yes	Class-separating projection	Requires labels
t-SNE	Unsupervised visualization	2D/3D coordinates	Usually yes	Reveals local structure	Not suitable as a general predictive transform

How to Explain Any Program in Viva

1. State the problem type and the target output.
2. Name the dataset and describe X and y.
3. Explain preprocessing and why it is needed.
4. Describe the train-test split or explain why clustering has no target split.
5. Name the model and its important parameters.
6. Explain fit(), predict() or fit_predict().
7. Interpret at least one evaluation metric.
8. State one limitation and one possible improvement.

Blank Practical Record Template

Record heading	Student entry
Experiment number and date	

Record heading	Student entry
Aim	
Software / requirements	
Theory	
Algorithm	
Program	
Output / screenshot	
Interpretation	
Result	
Teacher signature	

Glossary

Term	Meaning
Feature	Input variable used by a model
Target / label	Value that supervised learning tries to predict
Training data	Records used to learn model parameters
Test data	Separate records used for final evaluation
Fit	Learn parameters from data
Predict	Generate an output using a fitted model
Hyperparameter	User-selected setting such as k, C or max_depth
Data leakage	Information from outside training data improperly influences learning
Pipeline	Ordered preprocessing and model steps treated as one estimator
Overfitting	Model learns training details but generalizes poorly
Underfitting	Model is too simple to represent important patterns
Classification	Prediction of a discrete category
Regression	Prediction of a continuous number
Clustering	Grouping unlabelled observations
Centroid	Mean position of a K-Means cluster
Component	New transformed dimension produced by PCA or another technique

References and Documentation

- scikit-learn User Guide and API Reference: <https://scikit-learn.org/stable/>
- DecisionTreeClassifier: <https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html>
- LinearRegression: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html
- KNeighborsClassifier: <https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html>

- SVC: <https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html>
- KMeans: <https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html>
- PCA, LDA and t-SNE: scikit-learn decomposition, discriminant analysis and manifold modules.
- SciPy distance functions: <https://docs.scipy.org/doc/scipy/reference/spatial.distance.html>

Academic practice: Students should run each program, save output screenshots, change at least one parameter and write their own interpretation rather than copying metric values without explanation.